

Analisi delle Vulnerabilità di Rete

Tecniche di Scanning, Exploitation e Firewalling con Iptables

NECKER

12 Dicembre 2025

Indice

1	Network Scanning e Nmap	2
1.1	Cos'è Nmap e a cosa serve	2
1.2	Analisi delle Tecniche di Scansione	3
1.2.1	TCP Connect Scan (-sT)	3
1.2.2	TCP SYN (Stealth) Scan (-sS)	4
1.2.3	Idle Scan (-sI) e IP Segmentation	5
2	Exploitation: vsftpd 2.3.4 Backdoor	7
2.1	Analisi Tecnica della Vulnerabilità	7
2.1.1	Il Codice Malevolo	7
2.1.2	Dettagli sul Funzionamento	7
2.2	Esecuzione dell'Attacco tramite Metasploit	8
2.2.1	Concetto Chiave: La Bind Shell	8
2.2.2	Sequenza Logica dell'Exploit	8
3	Man In The Middle (MITM): ARP Poisoning	10
3.1	Il Protocollo ARP e la sua Vulnerabilità	10
3.2	Dinamica dell'Attacco	10
3.2.1	IP Forwarding	10
3.3	Visualizzazione dell'Attacco	11
4	Firewalling su Linux: Architettura Netfilter/Iptables	11
4.1	Netfilter	11
4.2	Iptables	11
4.3	Struttura Gerarchica e Funzione	12
4.3.1	1. Le Tabelle (Tables)	12
4.3.2	2. Le Catene (Chains)	12
4.3.3	3. Le Regole e i Target	13
4.4	Analisi Dettagliata del Flusso dei Pacchetti	13
4.5	Connection Tracking (Stateful Firewall)	14

1 Network Scanning e Nmap

Il primo passo in un'analisi di sicurezza o in un attacco è la fase di *Reconnaissance* (ricognizione). Lo strumento standard per questa operazione è **Nmap** (Network Mapper).

1.1 Cos'è Nmap e a cosa serve

Nmap è un tool open-source utilizzato per l'esplorazione della rete e l'auditing di sicurezza. Funziona inviando pacchetti "raw" (grezzi, manipolati a basso livello) ai sistemi target e analizzando le risposte. Le sue funzioni principali includono:

- **Host Discovery:** Identificare quali host sono attivi sulla rete (es. ping sweep).
- **Port Scanning:** Enumerare le porte aperte su un host target per capire quali servizi sono in ascolto.
- **Service & OS Detection:** Interrogare le porte aperte per determinare la versione del servizio in esecuzione e il sistema operativo dell'host (fingerprinting).

L'obiettivo dell'attaccante durante lo scanning è ottenere informazioni senza essere rilevato. La scelta del tipo di scansione dipende quindi dal compromesso tra **affidabilità** dei risultati e **stealthiness** (capacità di non essere tracciati dai sistemi di difesa come IDS o Firewall).

1.2 Analisi delle Tecniche di Scansione

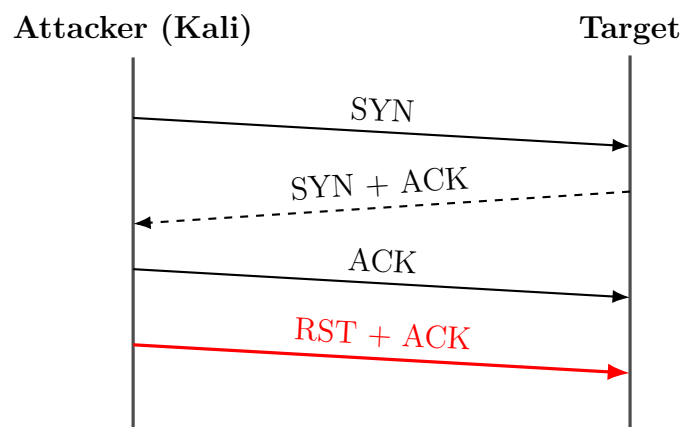
Di seguito vengono analizzate le tre tecniche principali trattate, evidenziando come gestiscono il protocollo TCP per evitare il rilevamento.

1.2.1 TCP Connect Scan (-sT)

Questa è la modalità di scansione di default quando l'utente che lancia Nmap **non ha privilegi di root**. Non potendo creare pacchetti raw (pacchetti costruiti manualmente accedendo alla scheda di rete), Nmap chiede al Sistema Operativo di stabilire una connessione completa usando la chiamata di sistema standard `connect()`.

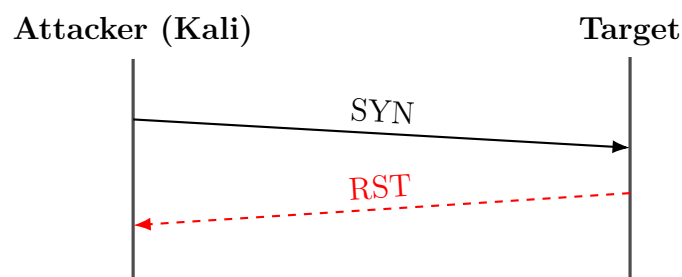
- **Funzionamento:** Viene completato l'intero *Three-Way Handshake* TCP.
- **Svantaggio (Rumorosità):** Poiché la connessione viene stabilita con successo, l'applicazione server (es. un Web Server Apache) accetta la connessione e molto probabilmente **scrive un log** dell'evento con l'IP dell'attaccante. Successivamente, Nmap chiude subito la connessione, ma la traccia è ormai lasciata.

Flusso dei pacchetti (Porta Aperta):



Nota: La connessione è aperta e subito abbattuta.

Flusso dei pacchetti (Porta Chiusa):

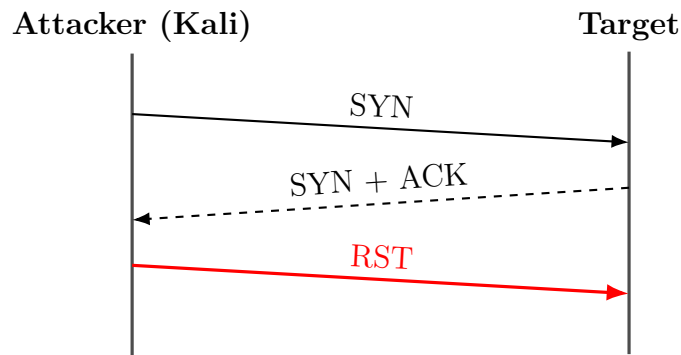


1.2.2 TCP SYN (Stealth) Scan (-sS)

Questa è la modalità di default se l'utente ha privilegi di **root**. È definita "Half-open scanning" perché la connessione non viene mai completata.

- **Funzionamento:** L'attaccante invia un SYN. Se riceve un SYN+ACK (indizio di porta aperta), non risponde con l'ACK finale (che chiuderebbe l'handshake), ma invia immediatamente un **RST** (Reset).
- **Vantaggio (Stealth):** Poiché l'handshake non si conclude, il livello applicativo del target spesso non viene mai notificato dell'apertura della connessione. Di conseguenza, **il servizio non genera log** dell'evento. È più difficile da rilevare rispetto al Connect Scan, anche se moderni Firewall e IDS possono identificarlo.

Flusso dei pacchetti (Porta Aperta):



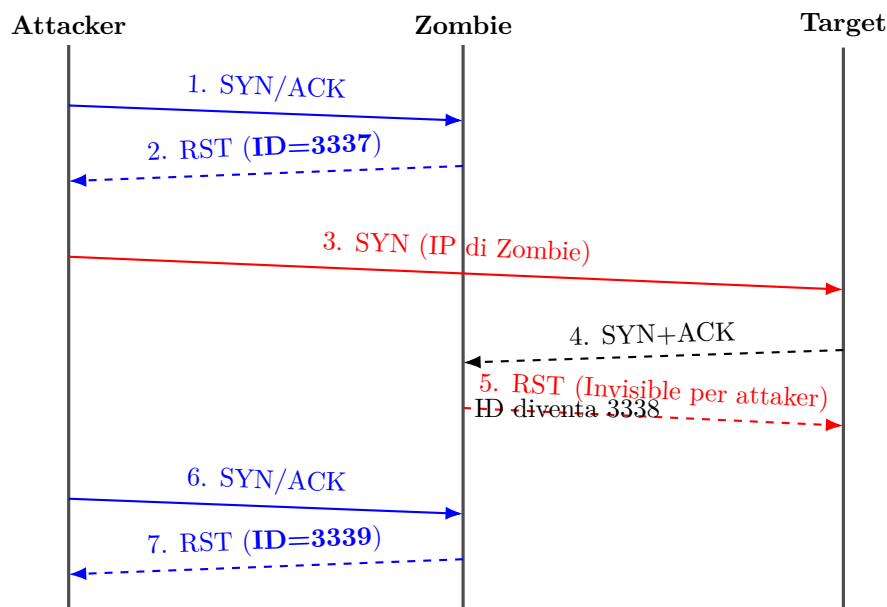
1.2.3 Idle Scan (-sI) e IP Segmentation

Questa è la tecnica più avanzata e furtiva ("Side-channel attack"). Permette di scansionare una vittima senza inviare alcun pacchetto con l'indirizzo IP dell'attaccante, utilizzando un host terzo detto "Zombie".

Il Concetto di IP ID Il protocollo IP prevede un campo **Identification** (16 bit) nell'header, usato per riassemblare i pacchetti frammentati. In alcuni sistemi operativi (o dispositivi legacy come stampanti), questo valore viene incrementato globalmente di 1 per ogni pacchetto inviato. L'attacco sfrutta questa prevedibilità per usare lo Zombie come oracolo.

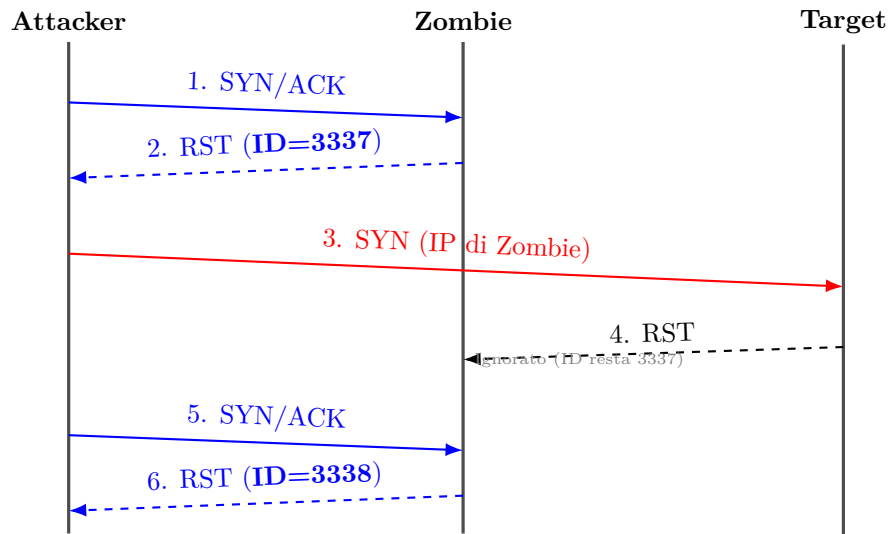
Algoritmo dell'Attacco L'attaccante vuole sapere se la porta della Vittima è aperta, ma vuole che la Vittima veda solo l'IP dello Zombie.

1. **Probe 1 (Campionamento iniziale):** L'attaccante invia un SYN/ACK allo Zombie. Lo Zombie, non avendo richiesto nulla, risponde con un RST. L'attaccante nota l'IP ID del pacchetto di risposta (es. $ID = 3337$).
2. **Solicitazione (Spoofing):** L'attaccante invia un SYN alla Vittima, ma **falsifica l'IP sorgente** mettendo quello dello Zombie.
 - **Scenario Porta APERTA:** La Vittima riceve il SYN e risponde con SYN+ACK allo Zombie. Lo Zombie (che non ha aperto connessioni) riceve il SYN+ACK e risponde alla Vittima con un RST. **Inviando questo RST, il contatore IP ID dello Zombie aumenta di 1** (diventa virtualmente 3338).



Risultato: $ID + 2$ (Porta APERTA)

- **Scenario Porta CHIUSA:** La Vittima risponde con RST allo Zombie. Lo Zombie ignora il pacchetto RST. Il contatore IP ID non cambia.



Risultato: ID + 1 (Porta CHIUSA)

Figura 1: Diagramma di sequenza: Idle Scan su porta CHIUSA

3. **Probe 2 (Verifica finale):** L'attaccante interroga di nuovo lo Zombie inviando un SYN/ACK. Lo Zombie risponde con un RST e il suo attuale IP ID.

Deduzione dello Stato Confrontando l'ID del Probe 1 (ID_1) e del Probe 2 (ID_2):

- **Porta Aperta ($\Delta ID = 2$):** Se $ID_2 = 3339$, significa che lo Zombie ha inviato 2 pacchetti (uno di risposta alla vittima + uno di risposta al secondo probe dell'attaccante).
- **Porta Chiusa ($\Delta ID = 1$):** Se $ID_2 = 3338$, lo Zombie ha inviato solo 1 pacchetto (la risposta al secondo probe dell'attaccante). La sollecitazione verso la vittima non ha causato reazioni.

2 Exploitation: vsftpd 2.3.4 Backdoor

Il 3 luglio 2011, il manutentore di vsftpd (Very Secure FTP Daemon: questo programma serve per creare un filesystem accessibile da remoto), Chris Evans, annunciò un grave incidente di sicurezza. L'archivio `.tar.gz` della versione **2.3.4**, ospitato sul sito ufficiale di distribuzione, era stato sostituito da un attaccante ignoto con una versione manomessa contenente una **backdoor**.

Questo evento rappresenta un classico esempio di *Supply Chain Attack*: invece di attaccare gli utenti finali, l'attaccante ha compromesso la fonte del software, infettando a cascata chiunque lo scaricasse.

2.1 Analisi Tecnica della Vulnerabilità

La modifica malevola non fu inserita nel codice principale di gestione delle connessioni, ma fu occultata all'interno di una funzione di utilità apparentemente innocua chiamata `str_contains_space`, situata nel file `str.c`.

Questa funzione dovrebbe limitarsi a controllare se una stringa contiene spazi, ma include una logica condizionale aggiuntiva ("Logic Bomb").

2.1.1 Il Codice Malevolo

Di seguito è riportato il codice C incriminato. L'attaccante ha sfruttato valori esadecimali per offuscare la stringa di attivazione.

```
1 // Funzione: str_contains_space
2 // Scopo originale: Controllare se l'username contiene spazi
3 for (i=0; i < p_str->len; i++)
4 {
5     // Se trova uno spazio, restituisce 1 (comportamento legittimo)
6     if (vsf_sysutil_isspace(p_str->p_buf[i]))
7     {
8         return 1;
9     }
10    // --- INIZIO BACKDOOR ---
11    // Controllo offuscato su due byte specifici
12    // 0x3a in esadecimale corrisponde al carattere ASCII ':' (due punti)
13    // 0x29 in esadecimale corrisponde al carattere ASCII ')' (parentesi
    chiusa)
14    else if((p_str->p_buf[i] == 0x3a) && (p_str->p_buf[i+1] == 0x29))
15    {
16        // Se la sequenza ":" viene trovata, viene lanciato il payload
17        vsf_sysutil_extra();
18    }
19    // --- FINE BACKDOOR ---
20 }
21 return 0;
22
```

2.1.2 Dettagli sul Funzionamento

1. **Il Trigger (Innesco):** La condizione `if` controlla se nell'username fornito durante il login appaiono consecutivamente i byte `0x3a` e `0x29`. In ASCII, questi corrispondono alla faccina sorridente `:)`.

2. **Il Payload (`vsf_sysutil_extra`):** Se il trigger viene attivato, viene chiamata la funzione `vsf_sysutil_extra()`. Nonostante il nome generico (scelto per non destare sospetti), questa funzione esegue una serie di chiamate di sistema per:

- Aprire un socket TCP.
- Mettersi in ascolto (bind) sulla porta **6200**.
- Eseguire una `exec1` per lanciare una shell di sistema (`/bin/sh`).

2.2 Esecuzione dell'Attacco tramite Metasploit

Per simulare l'attacco si può usare **Metasploit Framework**. Metasploit è una piattaforma modulare per il penetration testing che funge da "coltellino svizzero": permette di selezionare un *exploit* (il codice che sfrutta la vulnerabilità), configurare un *payload* (l'azione da eseguire dopo l'accesso) e lanciare l'attacco verso il target (Metasploitable 2).

2.2.1 Concetto Chiave: La Bind Shell

L'attacco alla backdoor di vsftpd 2.3.4 è un esempio classico di **Bind Shell**. A differenza di una *Reverse Shell* (dove la vittima si connette all'attaccante), in questo caso è la vittima ad aprire una nuova porta (la 6200) e mettersi in ascolto. L'attaccante deve quindi avviare una seconda connessione verso questa porta per accedere alla riga di comando.

2.2.2 Sequenza Logica dell'Exploit

L'attacco si svolge in quattro fasi distinte, gestite automaticamente dal modulo `exploit/unix/ftp/vsftpd_234_backdoor`:

1. Fase 1: Handshake e Trigger (Porta 21)

- L'attaccante stabilisce una normale connessione TCP sulla porta 21 (Command Channel del protocollo FTP).
- Durante la fase di autenticazione, invia un username malevolo contenente la sequenza di attivazione: `USER user:).`
- Invia una password qualsiasi (es. `PASS pass`), necessaria per completare il ciclo di input che spinge il server a processare l'username.

2. Fase 2: Esecuzione della Logic Bomb

- Il processo `vsftpd` sulla vittima analizza l'username tramite la funzione infetta `str_contains_space`.
- Rilevata la faccina `:`, il flusso del programma viene dirottato verso la funzione `vsf_sysutil_extra()`.
- Questa funzione esegue una *system call* per eseguire `/bin/sh` (la shell di sistema), "legandola" (binding) alla porta TCP 6200.

3. Fase 3: Connessione al Payload (Porta 6200)

- Metasploit, dopo aver inviato il trigger, attende brevemente e tenta di connettersi alla porta 6200 della vittima.

- Se la connessione ha successo, non viene presentato un prompt di login: l'attaccante è direttamente collegato allo standard input/output della shell di sistema.

4. Fase 4: Privilege Escalation (Root Access)

- **Perché siamo Root?** Normalmente, un server FTP "droppa" (abbandona) i privilegi di root dopo l'avvio per sicurezza. Tuttavia, la vulnerabilità si trova nel codice che gestisce l'autenticazione, una fase molto precoce in cui il demone ha ancora bisogno dei privilegi di amministratore (per esempio, per accedere ai file delle password o bindare la porta 21).
- **Conseguenza:** La shell generata eredita i permessi del processo padre. Poiché il padre era root in quel momento, la shell è root. L'attaccante ha il controllo totale del sistema.

3 Man In The Middle (MITM): ARP Poisoning

L'attacco **ARP Poisoning** (o ARP Spoofing) è una tecnica che permette a un attaccante di posizionarsi logicamente tra due host comunicanti in una rete locale (LAN), intercettando, leggendo o modificando tutto il traffico che transita tra di loro.

Per comprendere l'attacco, è necessario analizzare la debolezza intrinseca del protocollo ARP.

3.1 Il Protocollo ARP e la sua Vulnerabilità

Il protocollo ARP (Address Resolution Protocol) serve a mappare un indirizzo IP (Livello 3) nel corrispondente indirizzo MAC (Livello 2), necessario per la consegna fisica dei frame Ethernet.

La vulnerabilità risiede nel fatto che ARP è un protocollo **stateless** (senza stato) e privo di autenticazione:

- I dispositivi accettano pacchetti *ARP Reply* anche se non hanno mai inviato una corrispondente *ARP Request*.
- Non esiste un meccanismo di verifica: se qualcuno dichiara "Io sono l'IP 192.168.1.1", gli altri dispositivi nella rete si fidano ciecamente e aggiornano la loro ARP Cache locale.

3.2 Dinamica dell'Attacco

L'attaccante (Kali) sfrutta questa fiducia inviando continuamente pacchetti ARP falsificati (detti *Gratuitous ARP*) alle due vittime.

Nello scenario del laboratorio, le entità coinvolte sono:

- **Attacker (Kali):** IP 192.168.206.65, MAC 00-15-50-01-02-07
- **Vittima 1 (Host/Gateway):** IP 192.168.201.216, MAC 00-15-50-01-02-06
- **Vittima 2 (Metasploitable):** IP 192.168.201.124, MAC 00-15-50-01-02-05

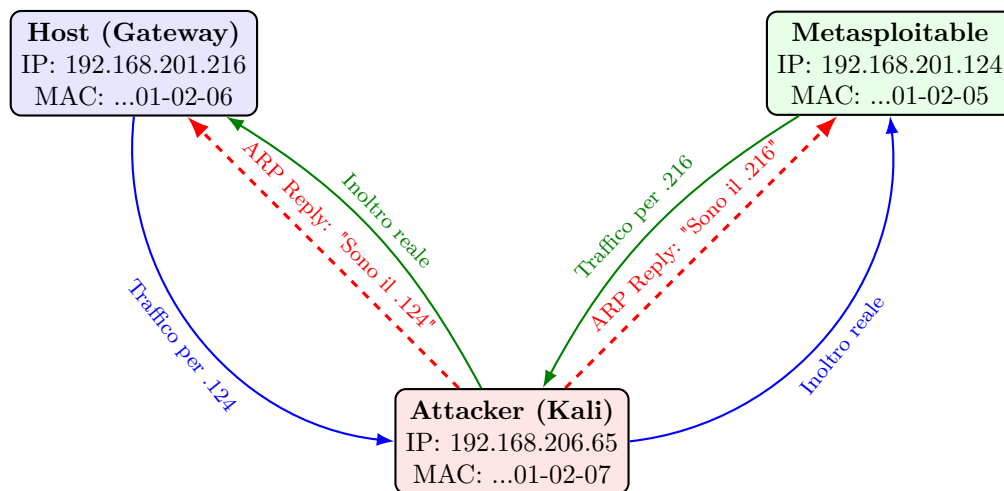
L'attacco avviene in due direzioni simultanee:

1. **Avvelenamento dell'Host:** L'attaccante invia un pacchetto all'Host dicendo:
"L'IP 192.168.201.124 (Metasploitable) si trova presso il MAC 00-15-50-01-02-07 (Kali)."
2. **Avvelenamento di Metasploitable:** L'attaccante invia un pacchetto a Metasploitable dicendo:
"L'IP 192.168.201.216 (Host) si trova presso il MAC 00-15-50-01-02-07 (Kali)."

3.2.1 IP Forwarding

Una volta che le cache ARP delle vittime sono "avvelenate", esse invieranno i pacchetti destinati all'altro host direttamente all'attaccante (a livello Ethernet). Affinché le vittime non si accorgano di nulla (e la connessione non cada), l'attaccante deve abilitare l'**IP Forwarding**. In questo modo, Kali agisce come un router trasparente: riceve i pacchetti, li analizza (sniffing con Wireshark o Ettercap) e li inoltra al vero destinatario.

3.3 Visualizzazione dell'Attacco



4 Firewalling su Linux: Architettura Netfilter/Iptables

Per comprendere il firewalling su Linux, è fondamentale distinguere tra due componenti che spesso vengono confusi: **Netfilter** e **iptables**.

4.1 Netfilter

Netfilter è un framework integrato direttamente all'interno del *Kernel Linux*. Immaginalo come una serie di "caselli doganali" o **Hooks** (ganci) posizionati strategicamente lungo i "tubi" dove scorrono i dati di rete del sistema operativo.

Ogni volta che un pacchetto dati entra nella scheda di rete o viene generato da un programma, deve obbligatoriamente passare attraverso questi ganci.

- Netfilter è "invisibile" all'utente.
- Lavora in **Kernel-space** (massima priorità e velocità).
- È l'entità che fisicamente ferma, modifica o lascia passare il pacchetto.

4.2 Iptables

Iptables è il programma a riga di comando che usi tu (amministratore).

- Lavora in **User-space**.
- Non tocca i pacchetti direttamente. Serve solo a "scrivere il manuale delle istruzioni" che Netfilter dovrà seguire.
- Quando lanci un comando **iptables**, stai essenzialmente dicendo al Kernel: *"Ehi Netfilter, al casello di Ingresso (INPUT), se vedi un pacchetto dalla porta 80, scartalo"*.

In sintesi: **iptables** è l'interfaccia di configurazione, mentre **Netfilter** è il motore che esegue il lavoro sporco all'interno dello stack TCP/IP.

4.3 Struttura Gerarchica e Funzione

Il sistema è organizzato secondo una rigida gerarchia:

Tabelle $\supset$ Catene (Chains) $\supset$ Regole

Lo scopo di questa struttura gerarchica è **separare le preoccupazioni** (Separation of Concerns). Ogni livello è responsabile di un aspetto diverso dell'elaborazione del pacchetto e include quello sotto:

1. **Tabelle:** Definiscono *cosa* si può fare al pacchetto (Filtrare? Modificare l'indirizzo? Alterare l'header?).
2. **Catene:** Definiscono *quando* (il punto nel flusso di rete) l'operazione deve avvenire.
3. **Regole:** Definiscono *come* trattare il pacchetto (i criteri e l'azione finale).

ES: filtriamo il traffico, in input, con un certo criterio.

4.3.1 1. Le Tabelle (Tables)

Le tabelle raggruppano le catene in base al *tipo* di manipolazione che è consentito. Le tre principali sono:

- **Filter (Default):** La tabella più comune, usata per prendere decisioni di **ammissione o blocco** (DROP/ACCEPT) dei pacchetti. Non modifica mai gli header del pacchetto.
- **Nat (Network Address Translation):** Usata per modificare gli indirizzi IP sorgente o destinazione (e le porte) del pacchetto. È essenziale per il *Masquerading* (SNAT) e il *Port Forwarding* (DNAT).
- **Mangle:** Usata per alterare campi specifici dell'header IP (es. TOS - Type of Service, o TTL - Time To Live) per scopi avanzati come Quality of Service (QoS) o debugging.

4.3.2 2. Le Catene (Chains)

Le catene sono **punti di controllo (hooks)** predefiniti e ordinati all'interno dello stack di rete del kernel. Un pacchetto, seguendo il suo percorso, attraversa sempre queste catene in un ordine fisso.

- **PREROUTING:** Il primo punto d'ingresso per i pacchetti in arrivo, prima che sia stata presa la decisione di routing. Qui si applica il DNAT (modifica dell'indirizzo di destinazione).
- **INPUT:** Contiene le regole per i pacchetti la cui destinazione è la *macchina firewall stessa* (localhost).
- **FORWARD:** Contiene le regole per i pacchetti destinati ad *attraversare* il firewall (la macchina agisce da router).
- **OUTPUT:** Contiene le regole per i pacchetti *generati localmente* dal firewall.
- **POSTROUTING:** L'ultimo punto di controllo prima che il pacchetto lasci la scheda di rete. Qui si applica l'SNAT/Masquerading.

4.3.3 3. Le Regole e i Target

Ogni catena è una lista ordinata di regole. Il pacchetto viene confrontato sequenzialmente con ogni regola. Se c'è una corrispondenza (match) con i criteri della regola, viene eseguita l'azione definita dal **Target** e l'elaborazione della catena può terminare.

I Target principali sono:

ACCEPT: Il pacchetto è accettato e prosegue nel suo percorso attraverso lo stack di rete.

DROP: Il pacchetto viene scartato silenziosamente. Il mittente non riceve alcuna notifica (effetto "buco nero", massima sicurezza).

REJECT: Il pacchetto viene scartato, ma viene inviato un messaggio di errore ICMP (Port Unreachable) al mittente. Utile per il debugging.

LOG: Scrive un messaggio nel registro di sistema (**syslog**) e continua alla regola successiva (è un target non terminante).

4.4 Analisi Dettagliata del Flusso dei Pacchetti

La posizione in cui una regola viene applicata è vitale, in quanto il pacchetto segue sempre lo stesso percorso predefinito.

Percorso A: Pacchetti destinati al Firewall (Localhost)

Questo percorso è per il traffico indirizzato alla macchina locale (es. un client SSH che si connette al firewall).

1. Ingresso fisico (NIC).
2. **PREROUTING** (Tabelle Mangle, Nat).
3. **Routing Decision:** Il kernel decide che l'IP destinazione è locale.
4. **INPUT** (Tabella Filter, punto critico per l'accesso locale).
5. Processo Locale: Il pacchetto viene consegnato all'applicazione (es. SSHd).

Percorso B: Pacchetti in Transito (Forwarding)

Questo percorso è per il traffico che attraversa il firewall (la macchina funge da router per una rete interna).

1. Ingresso fisico (NIC).
2. **PREROUTING** (Tabelle Mangle, Nat per DNAT).
3. **Routing Decision:** Il kernel decide che l'IP destinazione è esterno e che il forwarding è abilitato.
4. **FORWARD** (Tabella Filter, punto critico per la sicurezza della LAN).
5. **POSTROUTING** (Tabelle Mangle, Nat per SNAT/Masquerading).
6. Uscita fisica (NIC WAN).

Percorso C: Pacchetti generati dal Firewall

Questo percorso è per il traffico originato dai processi locali del firewall (es. un ping o un aggiornamento).

1. Processo Locale (Applicazione genera il pacchetto).
2. **OUTPUT** (Tabella Filter, punto critico per le restrizioni sul traffico in uscita).
3. **Routing Decision**: Il kernel decide l'interfaccia di uscita.
4. **POSTROUTING** (Tabelle Mangle, Nat).
5. Uscita fisica (NIC).

4.5 Connection Tracking (Stateful Firewall)

Una delle funzionalità più potenti di Netfilter è il modulo **conntrack**, che trasforma Iptables da un **firewall stateless** (regole applicate ai singoli pacchetti, non ricordano i pacchetti precedenti) a un **firewall stateful** (regole applicate a flussi di pacchetti, ragionano sul contesto **es**: tipo di connessione, hanno memoria del contesto grazie a una tabella con tutte le connessioni attive). Invece di analizzare i pacchetti isolatamente, **conntrack** monitora lo stato dell'intera connessione.

Questo permette di scrivere regole basate sullo stato della connessione:

- **NEW**: Il pacchetto sta tentando di aprire una nuova connessione (es. il primo pacchetto SYN di TCP).
- **ESTABLISHED**: Il pacchetto appartiene a una connessione legittimamente aperta e attiva (il traffico di ritorno o il flusso di dati continuo).
- **RELATED**: Il pacchetto è correlato a una connessione esistente (es. una connessione dati aperta dopo un canale di controllo FTP).

Il grande **vantaggio** è la semplificazione della sicurezza: una singola regola come **Accetta tutto se state == ESTABLISHED** gestisce in modo sicuro tutto il traffico di risposta, consentendo solo l'apertura esplicita di nuove connessioni (**NEW**) necessarie.